

Linguagem C++

Módulo 2 · Vetores e matrizes

Prof. Antonio Marcos Selmini

● O que veremos neste módulo

1

Vetores estáticos

Declaração, índices, referências, percurso e uso em funções

2

Matrizes estáticas

Linhas, colunas e laços aninhados

3

Vetores dinâmicos

A classe `std::vector`: tamanho definido em tempo de execução

4

Matrizes dinâmicas

Vetores de vetores e matrizes irregulares

5

Boas práticas

Erros comuns e quando usar cada abordagem

1

Vetores estáticos

Armazenando vários valores do mesmo tipo em uma única variável

● O que é um vetor?

- Um vetor (array) é uma sequência de elementos do mesmo tipo, armazenados lado a lado na memória.
- Cada elemento é identificado por um índice inteiro, que começa em 0.
- Em um vetor de n posições, o último índice é $n - 1$.
- Todo o vetor é acessado por um único nome; o índice seleciona o elemento.

```
int notas[5] = {8, 7, 10, 6, 9};
```

8	7	10	6	9
0	1	2	3	4

notas[2] vale 10 · notas[4] é o último

Atenção notas[5] não existe. O C++ não verifica limites: acessar essa posição lê ou escreve em memória indevida e produz comportamento imprevisível.

● Declaração e inicialização

```
int v[5];           // sem valores
int a[5] = {1, 2, 3, 4, 5};
int b[] = {10, 20, 30}; // tamanho 3
int c[5] = {1, 2};     // 1 2 0 0 0
int d[5] = {};         // zeros

const int N = 100;
double x[N];           // OK

int n;
cin >> n;
int y[n];              // NÃO é C++ padrão!
```

C++

- O tamanho fica entre colchetes e deve ser uma constante conhecida em tempo de compilação.
- Quando a lista de valores é fornecida, o tamanho pode ser omitido.
- Posições não listadas recebem zero; sem lista, contêm lixo de memória.
- Para tamanhos definidos pelo usuário, usaremos `std::vector` (seção 3).

● Acesso e percurso com for

```
int v[5] = {4, 8, 15, 16, 23};

v[0] = 42;           // altera o 1º
cout << v[4];        // imprime 23

for (int i = 0; i < 5; i++) {
    cout << v[i] << " ";
}
// 42 8 15 16 23

int tam = sizeof(v) / sizeof(v[0]);
// tam = 5
```

C++

- O laço for é a forma clássica de percorrer um vetor: o contador *i* vai de 0 até tamanho – 1.
- A condição *i* < 5 (e não *i* <= 5) evita ultrapassar o último índice.
- `sizeof(v) / sizeof(v[0])` calcula o número de elementos de um vetor estático.

Dica Guarde o tamanho em uma constante (`const int N = 5`) e use *N* no laço e na declaração.

● O que é uma referência (&)?

```
int a = 10;
int& r = a;    // r é apelido de a

r = 25;        // altera a
cout << a;     // 25

a = 7;
cout << r;     // 7

int& x;        // ERRO: precisa
               // de inicialização
```

C++

Por que aprender agora? O & aparecerá no range-for e nos parâmetros de funções deste módulo.

- Uma referência é um segundo nome (apelido) para uma variável que já existe.
- Declara-se colocando & após o tipo: `int& r = a;`
- Não é uma cópia: r e a são a mesma posição de memória. Alterar uma altera a outra.
- Deve ser inicializada na declaração e, depois disso, sempre se refere à mesma variável.
- Neste contexto, & não é o operador lógico E (&&) nem um cálculo — é parte da declaração.

● Referências como parâmetros de função

```
// por valor: recebe uma CÓPIA
void tentaDobrar(int n) { n *= 2; }

// por referência: usa o ORIGINAL
void dobrar(int& n) { n *= 2; }

int main() {
    int x = 5;
    tentaDobrar(x);
    cout << x;    // 5 (não mudou)
    dobrar(x);
    cout << x;    // 10
}
```

C++

Passagem por valor

- A função recebe uma cópia; alterações ficam restritas a ela.

Passagem por referência (&)

- A função trabalha sobre a variável original; alterações permanecem após o retorno. Também evita copiar dados grandes.

Referência constante (const T&)

- Sem cópia e sem permissão para alterar: ideal para apenas ler.

● Percorrendo com o range-for

```
int v[5] = {4, 8, 15, 16, 23};

// leitura: x é uma cópia
for (int x : v) {
    cout << x << " ";
}

// alteração: x& referencia o
// elemento original
for (int& x : v) {
    x = x * 2;
}
// v = 8 16 30 32 46
```

C++

- Disponível desde o C++11: percorre todos os elementos sem índice.
- Lê-se “para cada x em v”.
- Por padrão, x recebe uma cópia; alterações não afetam o vetor.
- Com `int& x`, x é uma referência ao elemento: cada iteração trabalha diretamente sobre o vetor — útil para modificar os valores.
- Use quando não precisar do índice; caso contrário, prefira o for clássico.

● Exemplo: notas de uma turma

```
const int N = 5;
double notas[N], soma = 0;

for (int i = 0; i < N; i++) {
    cout << "Nota " << i + 1 << ": ";
    cin >> notas[i];
    soma += notas[i];
}

double maior = notas[0];
for (int i = 1; i < N; i++) {
    if (notas[i] > maior)
        maior = notas[i];
}

cout << "Média: " << soma / N;
cout << " Maior: " << maior;
```

Padrões clássicos

Acumulador (soma) atualizado a cada iteração.

Maior valor: inicia com o primeiro elemento e compara com os demais.

Pergunta Por que o segundo laço começa em $i = 1$ e não em $i = 0$?

● Vetores como parâmetros de função

```
void mostrar(int v[], int n) {
    for (int i = 0; i < n; i++)
        cout << v[i] << " ";
}

void zerar(int v[], int n) {
    for (int i = 0; i < n; i++)
        v[i] = 0;    // altera o original
}

int main() {
    int dados[4] = {3, 1, 4, 1};
    mostrar(dados, 4);    // 3 1 4 1
    zerar(dados, 4);
    mostrar(dados, 4);    // 0 0 0 0
}
```

- O tamanho não é conhecido pela função: passe-o como parâmetro.
- Diferentemente de variáveis simples, o vetor não é copiado — a função trabalha sobre os elementos originais.
- Consequência: alterações feitas dentro da função permanecem após o retorno.
- Para impedir alterações, declare o parâmetro como `const int v[]`.

2

Matrizes estáticas

Vetores com duas dimensões: linhas e colunas

● O que é uma matriz?

- Uma matriz é um vetor de duas dimensões: os elementos são organizados em linhas e colunas.
- Cada elemento é identificado por dois índices: `m[linha][coluna]`, ambos iniciando em 0.
- Uma matriz 3×4 tem 3 linhas (0 a 2) e 4 colunas (0 a 3), totalizando 12 elementos.
- Na memória, as linhas são armazenadas uma após a outra.

linhas ↓ colunas →

	0	1	2	3
0	2	7	1	8
1	2	8	1	8
2	2	8	4	5

`m[1][2]` vale 1

● Declaração e inicialização de matrizes

```
int m[3][4];           // 3 lin × 4 col

int a[2][3] = {
    {1, 2, 3},         // linha 0
    {4, 5, 6}          // linha 1
};

int z[2][3] = {};      // tudo zero

a[1][2] = 60;          // linha 1, col 2
cout << a[0][1];       // 2
```

C++

- O primeiro colchete indica o número de linhas; o segundo, o de colunas.
- Na inicialização, cada linha é um conjunto de chaves internas.
- Como nos vetores, ambos os tamanhos devem ser constantes de compilação.
- Uma boa prática é definir `const int L = 3, COL = 4;` e usar as constantes em toda a lógica.

● Percorrendo uma matriz com laços aninhados

```
const int L = 2, COL = 3;
int m[L][COL];

// leitura
for (int i = 0; i < L; i++)
    for (int j = 0; j < COL; j++)
        cin >> m[i][j];

// impressão em formato de tabela
for (int i = 0; i < L; i++) {
    for (int j = 0; j < COL; j++)
        cout << m[i][j] << "\t";
    cout << endl;
}
```

- O laço externo percorre as linhas (i); o interno, as colunas (j).
- Para cada valor de i, o laço interno executa por completo: $2 \times 3 = 6$ iterações no total.
- O endl após o laço interno quebra a linha ao terminar cada linha da matriz.
- \t (tabulação) alinha as colunas na tela.

● Exemplo: soma das linhas e diagonal

```
const int N = 3;
int m[N][N] = {{1,2,3},{4,5,6},{7,8,9}};
// soma de cada linha
for (int i = 0; i < N; i++) {
    int soma = 0;
    for (int j = 0; j < N; j++)
        soma += m[i][j];
    cout << "Linha " << i << ": "
         << soma << endl;
}
// diagonal principal: i == j
int diag = 0;
for (int i = 0; i < N; i++)
    diag += m[i][i];
cout << "Diagonal: " << diag;  // 15
```

Saída

Linha 0: 6
Linha 1: 15
Linha 2: 24
Diagonal: 15

Observe A variável soma é declarada dentro do laço externo: ela é reiniciada a cada nova linha.

● A limitação dos vetores e matrizes estáticos

O problema

O tamanho precisa ser conhecido ao compilar. Se o programa só descobre a quantidade de dados durante a execução (lendo do usuário ou de um arquivo), é preciso reservar um máximo — desperdiçando memória ou, pior, faltando espaço.

A solução sem ponteiros

A biblioteca padrão oferece `std::vector`, um contêiner que cresce e diminui em tempo de execução, conhece o próprio tamanho e libera a memória automaticamente.

Para mais adiante Também é possível alocar memória manualmente com `new` e `delete`, mas isso exige ponteiros — assunto de um módulo futuro. Por ora, `std::vector` resolve todos os casos.

3

Vetores dinâmicos

A classe `std::vector` da biblioteca padrão

● Apresentando std::vector

```
#include <vector>
using namespace std;

vector<int> a;           // vazio
vector<int> b(5);        // 5 zeros
vector<double> c(4, 1.5); // 4 × 1.5
vector<int> d = {3, 1, 4};

int n;
cin >> n;
vector<int> e(n);        // tamanho
                        // em execução!
```

C++

- Requer #include <vector>.
- O tipo dos elementos vai entre < >: vector<int>, vector<string>...
- O tamanho pode ser uma variável comum, definida durante a execução.
- O acesso por índice é idêntico ao vetor estático: e[0], e[i].
- A memória é gerenciada automaticamente — nada a liberar.

● Principais operações

Operação	O que faz
<code>v.push_back(x)</code>	Acrescenta x ao final (o vetor cresce)
<code>v.size()</code>	Número atual de elementos
<code>v[i]</code> / <code>v.at(i)</code>	Acesso ao elemento i (at verifica o limite e lança erro)
<code>v.front()</code> / <code>v.back()</code>	Primeiro e último elementos
<code>v.pop_back()</code>	Remove o último elemento
<code>v.empty()</code>	true se não há elementos
<code>v.clear()</code>	Remove todos os elementos

Dica `v.size()` devolve um valor sem sinal; nos laços, use `size_t i` ou `int i` com conversão: `for (int i = 0; i < (int) v.size(); i++)`.

● Exemplo: quantidade desconhecida

```
vector<int> v;
int x;

cout << "Valores (-1 encerra): ";
cin >> x;
while (x != -1) {
    v.push_back(x);
    cin >> x;
}

cout << v.size() << " lidos" << endl;
cout << "Último: " << v.back() << endl;

for (int i = v.size() - 1; i >= 0; i--)
    cout << v[i] << " ";    // invertido
```

- O vetor começa vazio e cresce a cada `push_back` — sem tamanho máximo definido pelo programador.
- Com vetor estático, seria preciso reservar um limite e controlar um contador à parte.
- `size()` elimina a necessidade de passar o tamanho separadamente.

Atenção `v.back()` em um vetor vazio é indefinido — verifique `v.empty()` antes.

● std::vector como parâmetro de função

```
// cópia: alterações não saem daqui
double media(vector<double> v) { ... }

// referência: altera o original
void dobrar(vector<int>& v) {
    for (int& x : v) x *= 2;
}

// referência constante: eficiente
// e protegida contra alterações
void mostrar(const vector<int>& v) {
    for (int x : v) cout << x << " ";
}
```

C++

- Ao contrário do vetor estático, um vector passado por valor é copiado inteiro — custoso para muitos elementos.
- O & no parâmetro (referência) evita a cópia e permite modificar o original.
- const vector<T>& é a forma recomendada para apenas ler: sem cópia e sem risco de alteração.
- Não é preciso passar o tamanho: a função usa v.size().

● Bônus: ordenando com <algorithm>

```
#include <algorithm>

vector<int> v = {5, 2, 9, 1, 7};

sort(v.begin(), v.end());
// 1 2 5 7 9

sort(v.begin(), v.end(),
     greater<int>());
// 9 7 5 2 1

int maior = *max_element(
    v.begin(), v.end());
```

C++

- begin() e end() delimitam o intervalo a ser processado — todo o vetor, no caso.
- greater<int>() inverte o critério para ordem decrescente.
- A mesma biblioteca oferece reverse, count, find, min_element e outras funções úteis.
- Funciona também com vetores estáticos: sort(v, v + n).

4

Matrizes dinâmicas

Vetores de vetores: dimensões definidas em tempo de execução

● Matriz como vetor de vetores

```
int lin, col;
cin >> lin >> col;

// lin linhas, cada uma com col zeros
vector<vector<int>> m(lin,
                    vector<int>(col));

m[1][2] = 7;           // acesso igual
cout << m.size();      // n° de linhas
cout << m[0].size();   // n° de colunas
```

C++

- `vector<vector<int>>` é um vetor cujos elementos são, eles mesmos, vetores de `int` — cada um representa uma linha.
- O construtor recebe o número de linhas e o “modelo” de cada linha: `vector<int>(col)`.
- `m.size()` é o número de linhas; `m[i].size()`, o de colunas da linha `i`.

Leitura `vector<vector<int>> m(lin, vector<int>(col))` lê-se: “`m` é um vetor de `lin` vetores, cada um com `col` inteiros”.

● Exemplo: leitura e soma das linhas

```
int lin, col;
cout << "Linhas e colunas: ";
cin >> lin >> col;
vector<vector<int>> m(lin,
                    vector<int>(col));
for (int i = 0; i < lin; i++)
    for (int j = 0; j < col; j++)
        cin >> m[i][j];

for (int i = 0; i < lin; i++) {
    int soma = 0;
    for (int x : m[i]) soma += x;
    cout << "Linha " << i
         << ": " << soma << endl;
}
```

- A lógica dos laços aninhados é exatamente a mesma da matriz estática.
- A diferença está na declaração: lin e col vêm do usuário.
- `m[i]` é um vetor comum, por isso pode ser percorrido com range-for.
- Em funções, receba a matriz como `const vector<vector<int>>& m`.

● Matrizes irregulares

```
vector<vector<int>> tri;  
  
for (int i = 1; i <= 4; i++) {  
    vector<int> linha(i, i);  
    tri.push_back(linha);  
}  
  
for (auto& l : tri) {  
    for (int x : l) cout << x << " ";  
    cout << endl;  
}
```

C++

Saída

```
1  
2 2  
3 3 3  
4 4 4 4
```

- Como cada linha é um vetor independente, nada obriga que todas tenham o mesmo número de colunas.
- Útil para triângulos, listas de adjacência de grafos e dados agrupados.
- `auto& l` deduz o tipo (`vector<int>&`) e evita copiar cada linha.



5

Boas práticas

Comparando abordagens e evitando erros comuns

● Vetor estático × `std::vector`

	Vetor estático <code>int v[N]</code>	<code>std::vector<int> v</code>
Tamanho	Fixo, constante de compilação	Definido e alterável em execução
Conhece o tamanho?	Não — precisa de variável extra	Sim — <code>v.size()</code>
Crescer/encolher	Impossível	<code>push_back</code> , <code>pop_back</code> , <code>resize</code>
Verificação de limite	Nenhuma	Opcional, com <code>v.at(i)</code>
Passagem a funções	Sem cópia; tamanho à parte	Por referência (&), sem cópia
Uso recomendado	Tamanhos pequenos e fixos	Caso geral

Recomendação Em C++ moderno, prefira `std::vector` por padrão; use vetores estáticos quando o tamanho é realmente fixo e pequeno.

● Erros comuns

Índice fora do limite

for ($i = 0$; $i \leq n$; $i++$) acessa $v[n]$, que não existe. Use $i < n$.

Tamanho variável em vetor estático

$\text{int } v[n]$ com n lido do teclado não é C++ padrão. Use $\text{vector<int> } v(n)$.

Esquecer `#include <vector>`

O compilador não reconhece `vector`. Inclua o cabeçalho.

Copiar vetor sem necessidade

Parâmetro $\text{vector<int> } v$ copia tudo. Use $\text{const vector<int>\& } v$.

Vetor sem inicialização

$\text{int } v[5]$; contém lixo. Use $\text{int } v[5] = \{\}$ ou $\text{vector<int> } v(5)$.

Trocar linha por coluna

$m[j][i]$ em vez de $m[i][j]$. Padronize: i para linhas, j para colunas.

● Exercícios propostos

- 1 Leia 10 números em um vetor e mostre-os na ordem inversa.
- 2 Leia n valores em um vetor e informe quantos são maiores que a média.
- 3 Dado um vetor, conte quantas vezes um valor informado pelo usuário aparece.
- 4 Leia uma matriz 3×3 e verifique se ela é simétrica ($m[i][j] == m[j][i]$).
- 5 Leia lin e col , preencha uma matriz dinâmica e mostre a maior soma de coluna.
- 6 Monte a matriz irregular do triângulo de Pascal com n linhas usando `vector<vector<int>>`.

Resumo do módulo

- Vetores guardam vários valores do mesmo tipo; índices começam em 0, sem verificação.
- Matrizes são vetores bidimensionais percorridos com laços aninhados.
- Vetores e matrizes estáticos exigem tamanho constante de compilação.
- `std::vector` define o tamanho em execução, cresce com `push_back` e conhece `size()`.
- `vector<vector<T>>` representa matrizes dinâmicas, inclusive irregulares.

Referência: STROUSTRUP, B. Programming: Principles and Practice Using C++. 3. ed. Addison-Wesley, 2024.